

Nama : Nindya Alif Romland

NIM : H1D024031

Shift KRS : A

Shift Baru : F

TUGAS JARINGAN SYARAF TIRUAN 1 PERTEMUAN 6

Perceptron

1. Import library

```
# Import library
import numpy as np
import matplotlib.pyplot as plt
```

- numpy digunakan untuk operasi matematika dan manipulasi array/matriks.
- matplotlib.pyplot digunakan untuk membuat visualisasi grafik decision boundary perceptron.

2. Pembuatan Class Perceptron

```
# Buat kelas Perceptron
class Perceptron:
    # Simpan learning rate dan max epoch dalam konstruktor
    def __init__(self, alpha=0.1, epoch=10):
        self.alpha = alpha
        self.epoch = epoch
```

Class Perceptron dibuat sebagai blueprint model perceptron menggunakan konsep Object-Oriented Programming (OOP). Pada constructor `__init__`, parameter `alpha` digunakan sebagai learning rate untuk mengatur besar perubahan bobot, sedangkan `epoch` digunakan untuk menentukan jumlah maksimal iterasi pelatihan.

3. Fungsi Weighted Sum dan Predict

```
# Fungsi menghitung nilai y_in atau net
def weighted_sum(self, X):
    return np.dot(X, self.w_[1:]) + self.w_[0]

# Fungsi menerapkan fungsi aktivasi bipolar
def predict(self, X):
    return np.where(self.weighted_sum(X) >= 0.0, 1,-1)
```

- Fungsi `weighted_sum()` digunakan untuk menghitung nilai net input atau y_{in} dengan melakukan perkalian antara input dan bobot, kemudian ditambahkan bias.
- Fungsi `predict()` digunakan untuk menerapkan fungsi aktivasi bipolar. Jika hasil `weighted sum` lebih besar atau sama dengan 0 maka output bernilai 1, sedangkan jika kurang dari 0 maka output bernilai -1.

4. Fungsi Plot Decision Boundary

```
# Fungsi membuat simulasi garis pemisah data
def plot_decision_boundary(self, X, t, epoch):

    # Membuat titik data input
    plt.scatter(X[:, 0], X[:, 1], c=t.ravel(), marker='o', edgecolors='k', cmap=plt.cm.RdYlBu)

    # Menentukan limit tampilan bidang grafik
    x_min, x_max = X[:, 0].min()-1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min()-1, X[:, 1].max() + 1

    # Membuat garis pemisah
    x_vals = np.linspace(x_min, x_max, 100)
    y_vals = -(self.w_[0] + self.w_[1] * x_vals) / self.w_[2]
    plt.plot(x_vals, y_vals, 'b-', label=f'Decision boundary (Epoch {epoch+1})')

    plt.xlim(x_min, x_max)
    plt.ylim(y_min, y_max)
    plt.title(f'Decision Boundary Pada Epoch {epoch+1}')
    plt.xlabel('X1')
    plt.ylabel('X2')
    plt.legend()
    plt.show()
```

Fungsi ini digunakan untuk membuat visualisasi garis pemisah (decision boundary) hasil pembelajaran perceptron.

- `plt.scatter()` digunakan untuk menampilkan titik data input berdasarkan targetnya.
- `plt.plot()` digunakan untuk menggambar garis decision boundary berdasarkan bobot dan bias terbaru setiap epoch.
- `plt.figure()` digunakan agar setiap epoch memiliki grafik tersendiri.
- `plt.xlim()` dan `plt.ylim()` digunakan untuk mengatur batas tampilan sumbu X dan Y.
- `plt.title()`, `plt.xlabel()`, dan `plt.ylabel()` digunakan untuk memberi judul serta nama sumbu pada grafik.
- `plt.legend()` digunakan untuk menampilkan keterangan atau label garis pada grafik sehingga pengguna dapat mengetahui garis decision boundary yang ditampilkan berasal dari epoch tertentu.

5. Fungsi Training Perceptron

```
# Fungsi utama Perceptron
def fit(self, X, t):
    # Inisialisasi bobot dan bias awal = 0
    self.w_ = np.zeros(1 + X.shape[1])

    # Menyimpan hasil pada HasilPerceptron.txt
    with open("HasilPerceptron.txt", "w") as f:
        f.write("Masalah OR dengan Perceptron\n")
        f.write("-----\n")
        f.write(f"Input : {X}\n")
        f.write(f"Target: {t}\n")
        f.write(f"Bobot awal : {self.w_[1:]}\n")
        f.write(f"Bias awal : {self.w_[0]}\n")
        f.write(f"Learning rate : {self.alpha}\n")
        f.write(f"Max Epoch : {self.epoch}\n")

    # Iterasi Perceptron(Epoch)
    for epoch in range(self.epoch):
        f.write(f"\nEpoch {epoch + 1}/{self.epoch}\n")
        f.write("-----\n")
        error = np.array([])

        # Iterasi setiap pasang matriks input dengan targetnya
        for xi, target in zip(X, t):
            # Periksa input dengan model Perceptron
            y_pred = self.predict(xi)

            # Periksa apakah output sesuai target, jika iya maka error = 0
            error = np.append(error, target - y_pred)

        # Modifikasi bobot dengan Delta Rule, jika error = 0 maka bobot tetap
        update = self.alpha * error[-1]

        # Modifikasi bobot w
        self.w_[1:] += update * xi

        # Modifikasi bias b
        self.w_[0] += update

        # Menuliskan hasil tiap iterasi input pada satu epoch
        f.write(f"Input: {xi}, Target: {target}, Predict: {y_pred}, Error: {error[-1]}, Bo

    # Simulasikan garis pemisah model Perceptron
    self.plot_decision_boundary(X, t, epoch)
```

Fungsi fit() merupakan fungsi utama yang digunakan untuk melakukan proses training pada model perceptron. Pada fungsi ini, program akan melakukan pelatihan data secara berulang hingga mencapai jumlah epoch yang telah ditentukan. Selama proses training, bobot dan bias akan terus diperbaiki berdasarkan error hasil prediksi agar model dapat menghasilkan output yang sesuai dengan target.

- `self.w_ = np.zeros(1 + X.shape[1])`
Digunakan untuk menginisialisasi bobot dan bias dengan nilai awal 0. `1 + X.shape[1]` berarti jumlah bobot mengikuti jumlah fitur input ditambah satu bias.
- `for epoch in range(self.epoch):`
Digunakan untuk melakukan pengulangan training sebanyak jumlah epoch yang telah ditentukan.
- `error = np.array([])`
Membuat array kosong untuk menyimpan nilai error pada setiap data training dalam satu epoch.
- `for xi, target in zip(X, t):`
Digunakan untuk mengambil pasangan data input (xi) dan target (target) secara bersamaan.

- `y_pred = self.predict(xi)`
Model melakukan prediksi output berdasarkan bobot saat ini menggunakan fungsi `predict()`.
- `error = np.append(error, target - y_pred)`
Menghitung error dari selisih antara target dan hasil prediksi.
- `update = self.alpha * error[-1]`
Menghitung nilai perubahan bobot menggunakan Delta Rule.
- `self.w_[1:] += update * xi`
Digunakan untuk memperbaiki bobot berdasarkan nilai error dan data input.
- `self.w_[0] += update`
Digunakan untuk memperbaiki nilai bias.
- `self.plot_decision_boundary(X, t, epoch)`
Digunakan untuk menampilkan grafik decision boundary pada setiap epoch agar perubahan garis pemisah dapat divisualisasikan.

6. Main Program Perceptron (Perceptron_or.py)

```
# Import library
import numpy as np
import Perceptron as p

# Inisialisasi input dan target
X = np.array([[1,1], [1,-1], [-1,1], [-1,-1]])
t = np.array([[1], [1], [1], [-1]])

# Pemanggilan model Perceptron
model = p.Perceptron(alpha=0.1, epoch=10)
model.fit(X, t)
```

Kode di atas merupakan program utama untuk menjalankan model perceptron pada kasus logika OR bipolar.

- `import numpy as np`
Digunakan untuk mengimpor library numpy yang digunakan dalam pembuatan array data input dan target.
- `import Perceptron as p`
Digunakan untuk mengimpor class Perceptron dari file Perceptron.py dan memberi alias p.
- `X = np.array([[1,1], [1,-1], [-1,1], [-1,-1]])`
Digunakan untuk membuat data input bipolar logika OR yang terdiri dari empat kombinasi input.
- `t = np.array([[1], [1], [1], [-1]])`
Digunakan untuk membuat target output dari operasi logika OR bipolar.
- `model = p.Perceptron(alpha=0.1, epoch=10)`

Digunakan untuk membuat objek model perceptron dengan learning rate 0.1 dan jumlah epoch 10.

- `model.fit(X, t)`

Digunakan untuk memulai proses training perceptron menggunakan data input dan target yang telah dibuat sebelumnya.

Backpropagation

1. Import library

```
# Import library
import numpy as np
import matplotlib.pyplot as plt
```

- `import numpy as np`
Digunakan untuk mengimpor library NumPy dengan alias np. Library ini digunakan untuk operasi matriks, array, perhitungan numerik, dan manipulasi data pada jaringan saraf tiruan.
- `import matplotlib.pyplot as plt`
Digunakan untuk mengimpor library Matplotlib dengan alias plt. Library ini digunakan untuk membuat grafik error selama proses training Backpropagation.

2. Pembuatan Class Backpropagation

```
# Buat kelas Backpropagation
class Backpropagation:
```

Class ini berfungsi sebagai blueprint atau rancangan utama jaringan saraf tiruan Backpropagation yang berisi parameter jaringan, fungsi aktivasi, proses forward propagation, backward propagation, proses training, dan visualisasi error.

3. Konstruktor (__init__)

```
# Simpan learning rate, epoch, dan target error dalam konstruktor serta inisialisasi bobot dan
def __init__(self, alpha, epoch, target_error):
    np.random.seed(42)

    self.alpha = alpha
    self.epoch = epoch
    self.target_error = target_error

    self.n_input = 2
    self.n_hidden = 2
    self.n_output = 1

    self.w_hidden = np.random.rand(self.n_input, self.n_hidden)
    self.b_hidden = np.random.rand(1, self.n_hidden)

    self.w_output = np.random.rand(self.n_hidden, self.n_output)
    self.b_output = np.random.rand(1, self.n_output)
```

Fungsi konstruktor digunakan untuk menyimpan parameter training, menentukan jumlah neuron, serta menginisialisasi bobot dan bias awal secara random.

Penjelasan tiap bagian:

- `np.random.seed(42)` digunakan agar nilai random selalu sama setiap program dijalankan.
- `self.alpha` menyimpan learning rate.
- `self.epoch` menyimpan jumlah epoch maksimum.
- `self.target_error` menyimpan target error minimum.
- `self.n_input`, `self.n_hidden`, dan `self.n_output` menentukan jumlah neuron pada tiap layer.
- `self.w_hidden` dan `self.w_output` digunakan untuk membuat bobot awal random.
- `self.b_hidden` dan `self.b_output` digunakan untuk membuat bias awal random.

4. Fungsi Aktivasi dan Turunan Aktivasi

```
# Fungsi menerapkan fungsi aktivasi sigmoid bipolar atau tanh
def bi_sigmoid(self, x):
    return np.tanh(x)

# Fungsi menerapkan turunan fungsi aktivasi sigmoid bipolar atau tanh (asumsi x = output sigmoid)
def deriv_bi_sigmoid(self, x):
    return 1-x**2
```

- Fungsi aktif

```
def bi_sigmoid(self, x):
```

```
    return np.tanh(x)
```

Digunakan untuk mengubah nilai input menjadi output dengan rentang -1 sampai 1 menggunakan fungsi `tanh()`.

- Fungsi turun

```
def deriv_bi_sigmoid(self, x):
```

```
    return 1-x**2
```

Digunakan untuk menghitung turunan fungsi aktivasi `tanh()` yang digunakan pada proses backward propagation.

5. Fungsi Plot Error

```
# Fungsi membuat simulasi perbaikan bobot dan bias
def plot_error(self, x, epoch):
    plt.plot(range(1, epoch + 1), x, linestyle='-', color='b', label='Error')

    final_error = x[-1]
    plt.annotate(f'Epoch {epoch}, Error: {final_error:.4f}',
                xy=(epoch, final_error),
                xytext=(epoch - len(x) * 0.2, final_error + 0.05),
                arrowprops=dict(facecolor='black', arrowstyle="->"),
                fontsize=10, color='red')

    plt.title('Perbaikan Error Setiap Epoch')
    plt.xlabel('Epoch')
    plt.ylabel('Sum Square Error(SSE)')
    plt.grid(True)
    plt.legend()
    plt.show()
```

Fungsi `plot_error()` digunakan untuk menampilkan grafik perubahan error selama proses training.

- plt.plot() digunakan untuk membuat grafik error terhadap epoch.
- plt.annotate() digunakan untuk memberikan informasi nilai error terakhir pada grafik.
- plt.title() digunakan untuk memberi judul grafik.
- plt.xlabel() dan plt.ylabel() digunakan untuk memberi nama sumbu grafik.
- plt.grid(True) digunakan untuk menampilkan garis bantu pada grafik.
- plt.legend() digunakan untuk menampilkan label grafik.
- plt.show() digunakan untuk menampilkan grafik ke layar.

6. Fungsi Training (fit)

```
# Fungsi utama Backpropagation
def fit(self, X, t):
    errors_per_epoch = []

    # Menyimpan hasil pada hasilBackpropagation.txt
    with open("hasilBackpropagation.txt", "w") as f:
        f.write("Masalah XOR dengan Backpropagation\n")
        f.write("-----\n")
        f.write(f"Input : \n{X}\n")
        f.write(f"Target : \n{t}\n\n")
        f.write(f"Bobot awal hidden layer : \n{self.w_hidden}\n\n")
        f.write(f"Bias awal hidden layer : \n{self.b_hidden}\n\n")
        f.write(f"Bobot awal output layer : \n{self.w_output}\n\n")
        f.write(f"Bias awal output layer : \n{self.b_output}\n\n")
        f.write(f"Learning rate : {self.alpha}\n")
        f.write(f"Max Epoch : {self.epoch}\n\n")
        # Iterasi Backpropagation(epoch)
        for epoch in range(self.epoch):
            f.write("-----\n")
            f.write(f"Epoch {epoch + 1}/{self.epoch}\n")
            f.write("-----\n")
            total_error = 0
            count = 1
            output = np.array([])

            # Iterasi setiap pasang matriks input dengan targetnya
            for xi, target in zip(X, t):
                f.write(f"Data ke-{count}\n")
                f.write("-----\n")
                f.write("----- Forward Propagation-----\n")
```

```
# Operasikan input dengan hidden layer
h_in = np.dot(xi, self.w_hidden) + self.b_hidden
f.write(f"Operasi input ke hidden layer: \n{h_in}\n\n")

# Aktivasi hasil operasi hidden layer dengan fungsi tanh
h = self.bi_sigmoid(h_in)
f.write(f"Aktivasi hidden layer: \n{h}\n\n")

# Operasikan hidden layer dengan output layer
y_in = np.dot(h, self.w_output) + self.b_output
f.write(f"Operasi hidden ke output layer: \n{y_in}\n\n")

# Aktivasi hasil operasi output layer dengan fungsi tanh
y = self.bi_sigmoid(y_in)
output = np.append(output, y)
f.write(f"Aktivasi output layer: \n{y}\n")
f.write("----- Backward Propagation-----\n")

# Hitung error output layer terhadap target
error = target - y
total_error += np.sum(error**2)
f.write(f"Error: \n{error}\n\n")

# Operasikan error dengan turunan dari aktivasi output layer
d_y = error * self.deriv_bi_sigmoid(y)
f.write(f"Delta output (d_y): \n{d_y}\n\n")

# Hitung error hidden layer
error_h = np.dot(d_y, self.w_output.T)
f.write(f"Error hidden layer (error_h): \n{error_h}\n\n")
```

```

# Operasikan error dengan turunan dari aktivasi hidden layer
d_h = error_h * self.deriv_bi_sigmoid(h)
f.write(f"Delta hidden layer (d_h):\n{d_h}\n\n")

# Perbaiki bobot dan bias output layer
self.w_output += np.dot(h.T, d_y) * self.alpha
f.write(f"Bobot output layer baru (w_output):\n{self.w_output}\n\n")
self.b_output += np.sum(d_y, axis=0, keepdims=True) * self.alpha
f.write(f"Bias output layer baru (b_output):\n{self.b_output}\n\n")

# Perbaiki bobot dan bias hidden layer
self.w_hidden += np.dot(xi.reshape(2,1), d_h) * self.alpha
f.write(f"Bobot hidden layer baru (w_hidden):\n{self.w_hidden}\n\n")
self.b_hidden += np.sum(d_h, axis=0, keepdims=True) * self.alpha
f.write(f"Bias hidden layer baru (b_hidden):\n{self.b_hidden}\n\n")
f.write("-----\n")
count += 1

# Hitung Sum Square Error(SSE) pada setiap epoch
average_error = total_error / len(X)
errors_per_epoch.append(average_error)
f.write(f"Output : {output.reshape(1,4)}\n")
f.write(f"Sum Square Error(SSE) epoch ke-{epoch + 1}: {average_error}\n")

# Cek kondisi berhenti, jika Sum Square Error(SSE) kurang dari target error atau max epoch terd
if average_error < self.target_error or epoch + 1 == self.epoch:
    f.write("-----\n")
    f.write(f"Pelatihan berhenti pada epoch ke-{epoch + 1} karena ")
    f.write(f"Sum Square Error(SSE) mencapai target.\n" if epoch + 1 != self.epoch else "max epo
    f.write(f"Bobot akhir hidden layer :\n{self.w_hidden}\n\n")
    f.write(f"Bias akhir hidden layer :\n{self.b_hidden}\n\n")
    f.write(f"Bobot akhir output layer :\n{self.w_output}\n\n")
    f.write(f"Bias akhir output layer :\n{self.b_output}")
    self.plot_error(errors_per_epoch, epoch + 1)
    break

```

Fungsi fit() merupakan fungsi utama dalam proses training Backpropagation. Pada fungsi ini dilakukan forward propagation, backward propagation, update bobot dan bias, perhitungan error, hingga proses penghentian training.

- `errors_per_epoch = []`
Digunakan untuk menyimpan nilai error pada setiap epoch.
- `for epoch in range(self.epoch):`
Digunakan untuk melakukan training sebanyak jumlah epoch yang ditentukan.
- `h_in = np.dot(xi, self.w_hidden) + self.b_hidden`, `h = self.bi_sigmoid(h_in)`, `y_in = np.dot(h, self.w_output) + self.b_output`, `y = self.bi_sigmoid(y_in)`
Digunakan untuk menghitung output jaringan mulai dari input layer, hidden layer, hingga output layer.
- `error = target-y`
Digunakan untuk menghitung selisih antara target dan output jaringan.
- `d_y = error * self.deriv_bi_sigmoid(y)`, `error_h = np.dot(d_y, self.w_output.T)`, `d_h = error_h * self.deriv_bi_sigmoid(h)`
Digunakan untuk menghitung delta output layer dan hidden layer.
- `self.w_output += np.dot(h.T, d_y) * self.alpha`, `self.b_output += np.sum(d_y, axis=0, keepdims=True) * self.alpha`, `self.w_hidden += np.dot(xi.reshape(2,1), d_h) * self.alpha`, `self.b_hidden += np.sum(d_h, axis=0, keepdims=True) * self.alpha`
Digunakan untuk memperbaiki bobot dan bias berdasarkan hasil error.
- `average_error = total_error / len(X)`
Digunakan untuk menghitung rata-rata Sum Square Error (SSE) setiap epoch.
- `if average_error < self.target_error or epoch + 1 == self.epoch:`

Training akan berhenti apabila error sudah mencapai target, atau jumlah epoch maksimum tercapai.

7. Main Program Backpropagation (Backpropagation_or.py)

```
# Import library
import numpy as np
import Backpropagation as b

# Inisialisasi input dan target
X = np.array([[1, 1], [1,-1], [-1, 1], [-1,-1]])
t = np.array([[1], [1], [1], [-1]])

# Pemanggilan model Backpropagation
model = b.Backpropagation(alpha=0.3, epoch=1000, target_error=0.001)
model.fit(X, t)
```

- `import numpy as np`
Digunakan untuk mengimpor library NumPy yang digunakan dalam pembuatan array input dan target.
- `import Backpropagation as b`
Digunakan untuk mengimpor class Backpropagation dari file Backpropagation.py dan memberi alias b.
- `X = np.array([[1, 1], [1,-1], [-1, 1], [-1,-1]])`
Digunakan untuk membuat data input XOR bipolar yang terdiri dari empat kombinasi input.
- `t = np.array([[1], [1], [1], [-1]])`
Digunakan untuk membuat target output dari operasi logika XOR bipolar.
- `model = b.Backpropagation(alpha=0.3, epoch=1000, target_error=0.001)`
Digunakan untuk membuat objek model Backpropagation dengan learning rate sebesar 0.3, jumlah maksimum epoch 1000, dan target error 0.001.
- `model.fit(X, t)`
Digunakan untuk memulai proses training jaringan saraf tiruan Backpropagation menggunakan data input dan target yang telah dibuat sebelumnya.